PRG 107 – Python Programming
Kristiania University of Applied Sciences

By Huamin Ren

Huamin.ren@kristiania.no

Outline II - Python language basics IIII

1. Data structure and sequences
   - Tuple
   - List
   - Dict
   - Set
2. Functions
   - Namespaces, scope and local functions
   - Returning multiple values
   - Lambda functions

Outline - Python language basics I

1. *Data structure and sequences*
   - Tuple
   - *List*
   - Dict
   - Set
2. Functions
   - Namespaces, scope and local functions
   - Returning multiple values
   - Lambda functions

1.2 List

- Lists are variable-length and their contents can be modified in-place
- important!
- List creation, adding and removing, concatenation, slicing, sorting

- Lists and tuples are semantically similar (though tuples cannot be modified) and can be used interchangeably in many functions.

1.2 List

- Create
- Add
- Remove
- Concatenating
- Combine
- Sorting

Insert vs. pop

insert is computationally expensive compared with append, because references to subsequent elements have to be shifted internally to make room for the new element.

- The inverse operation to insert is pop, which removes and returns an element at a particular index

Elements can be removed by value with ***remove***, which locates the first such value and removes it from the list

Concatenating and combining lists

- Similar to tuples, adding two lists together with + concatenates them
- If you have a list already defined, you can append multiple elements to it using the extend method

+ vs. extend?

- Slicing

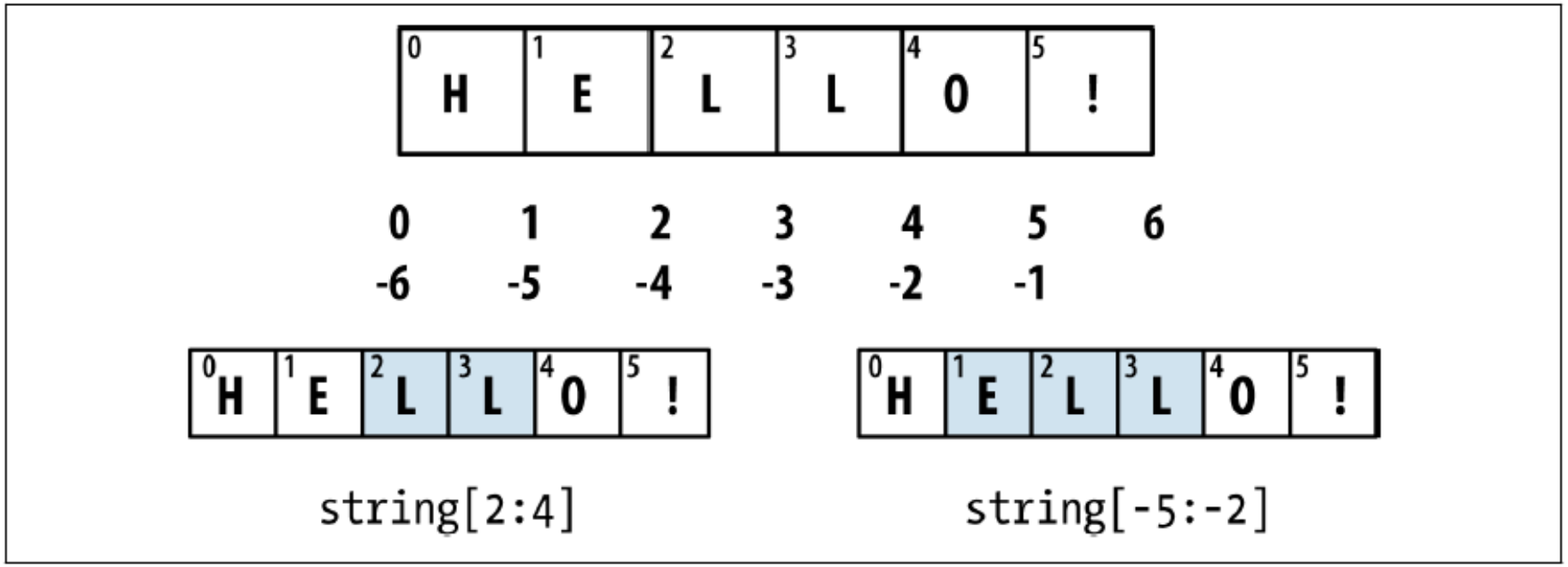

Figure 3-1. Illustration of Python slicing conventions

Outline - Python language basics I

1. *Data structure and sequences*
   - Tuple
   - List
   - *Dict*
   - Set
2. Functions
   - Namespaces, scope and local functions
   - Returning multiple values
   - Lambda functions

1.3 Dict

- It is a flexibly sized collection of *key-value* pairs, where key and value are Python objects
- Dictionary1 = {‘student_001’: ‘Lukas’, ‘student_002’: ‘Oscar’,…}
- Dictionary1[‘student_001’]

```
empty_dict = {}
d1 = {'a' : 'some value', 'b' : [1, 2, 3, 4]}
d1  d1: {'a': 'some value', 'b': [1, 2, 3, 4], 7: 'a
```

- Access
- Insert
- Set

```
d1[5] = 'some value'
d1
d1['dummy'] = 'another value'
d1
del d1[5]
d1
ret = d1.pop('dummy')
ret
d1
```

The update method changes dicts in-place, so any existing keys in the data passed to update will have their old values discarded.

1.4 Set

- A set is an unordered collection of unique elements - like dicts, but keys only, no values.

Build-in sequence functions

- Enumerate
- Zip

Python has a built-in function, enumerate, which returns a sequence of (i, value) tuples:

for i, value in enumerate(collection):

```
seq1 = ['foo', 'bar', 'baz']
seq2 = ['one', 'two', 'three']
zipped = zip(seq1, seq2)
ls= list(zipped)
print(ls)
```

```
[('foo', 'one'), ('bar', 'two'), ('baz', 'three')]
```

Outline - Python language basics I

1. Data structure and sequences
   - List
   - Dict
   - Set
2. *Functions*
   - Namespaces, scope and local functions
   - Returning multiple values
   - Lambda functions

2.1 Namespaces

Functions can access variables in two different scopes: global and local

Any variables assigned within a function by default are assigned to the local namespace. The local namespace is created when the function is called and immediately populated by the function's arguments

Typically global variables are used to store some kind of state in a system

2.2 Returning multiple values

```
return_value = f()
```

```
def f():
    a = 5
    b = 6
    c = 7
    return {'a' : a, 'b' : b, 'c' : c}
```

2.3 Lambda functions

- Writing functions consisting of a single statement, the result of which is the return value

```
def short_function(x):
    return x * 2

equiv_anon = lambda x: x * 2
```